COL362/632 Introduction to Database Management Systems

Query Processing – Joins

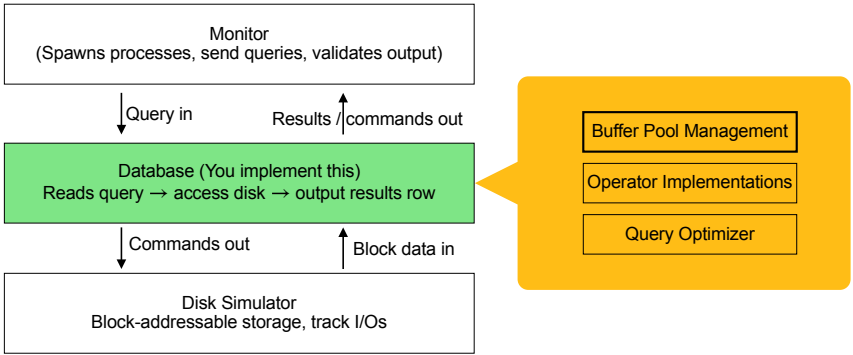
Kaustubh Beedkar

Department of Computer Science and Engineering
Indian Institute of Technology Delhi

Next Assignment (My Awesome DB)

Next assignment will be released later today!

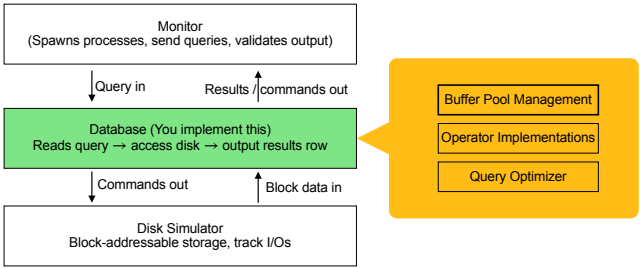
- Overall implement a toy DB in Rust



My Awesome DB

Constraints

- Limited Memory (at least 64MB)
- Single-threaded
- No file I/O, only use the disk simulator (scratch space)
- Time limit (more details later)



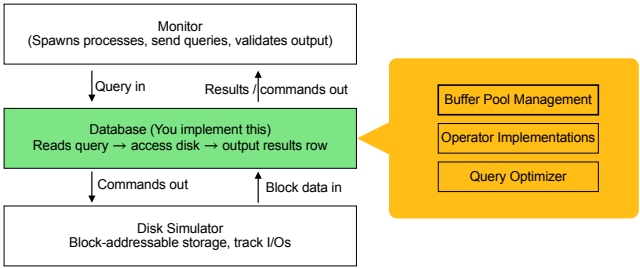
My Awesome DB

Query Specs

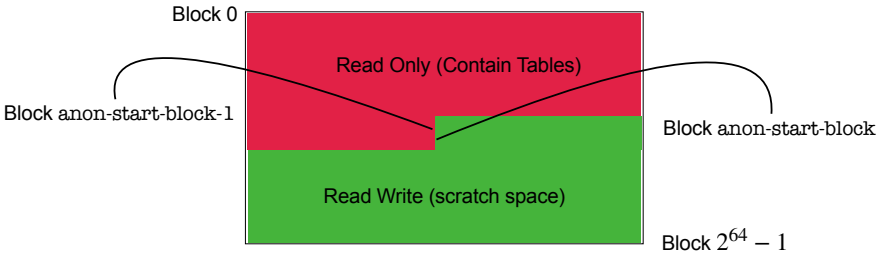
- Parser given for parsing a query into an AST comprising
  - Scan, Cross, Filter, Project, and Sort

Data Types

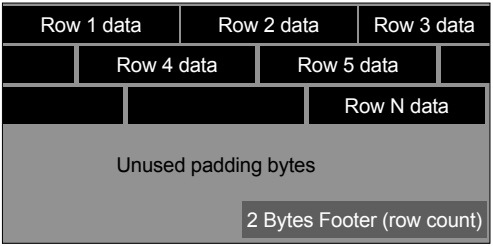
- Int32, Int64, Float32, Float64, String



Disk Simulator

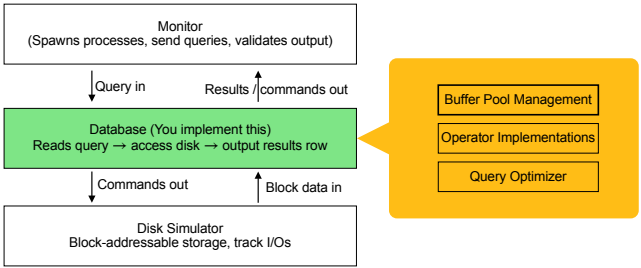


Block Layout



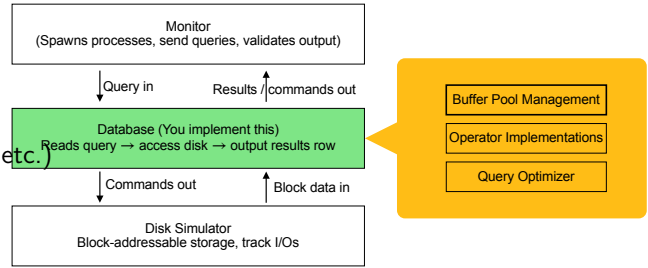
My Awesome DB

- Start early!
- Write simple programs to read/write blocks
- Implement a memory management strategy (LRU, CLOCK, MRU, ... be creative)
- Implement operators (scans, filters, projections, sort, and joins)
  - Implement different algorithms
- Implement an optimizer



Evaluation and Grading

- Scoring based on
  - simulated disk I/O time
    - Tracked by the disk simulator
    - Based on access patters (sequential/random access, tx rates, etc.)
  - CPU time of your process



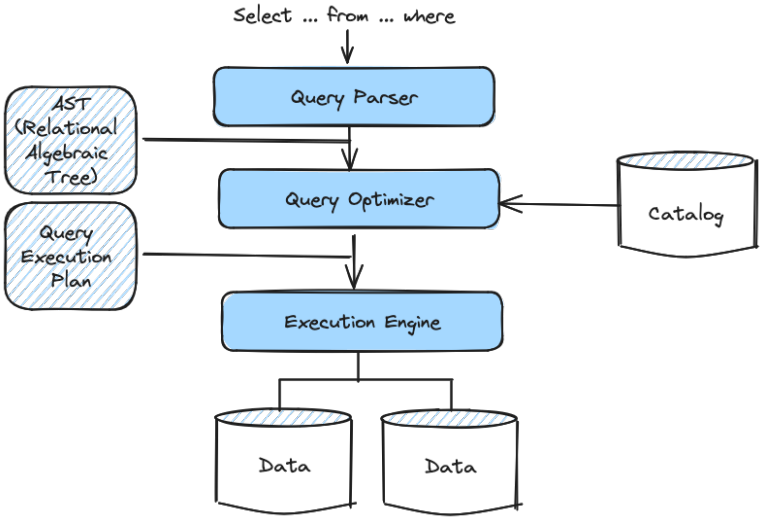
Evaluation and Grading

- Tiered System based on relative performance on a class leaderboard

Tier	Requirement
Bronze	Correct output, terminate within time limit
Silver	Relative performance ranking
Gold	Relative performance ranking
Platinum	Relative performance ranking
Diamond	Relative performance ranking

- LLM usage – Be my guest :) but won't get beyond bronze or silver
- Be creative... the entire play ground is yours
- First submission 13th April (mandatory)
- Subsequently can re-submit multiple times until 27th April to improve leaderboard rankings

Query Processing (Overview)



Outline

Joins

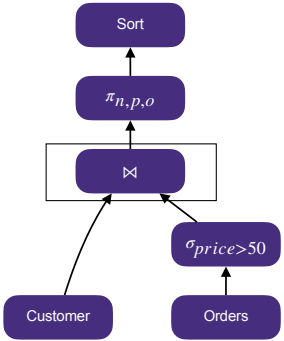
$R \bowtie_\theta S$

- We will assume equijoins, i.e., $\theta \equiv \{ R.A = S.A \}$
- Equijoin algorithms can be extented to support other joins
- R is outer relation
- S is inner relation

$R \bowtie S$ is one of the most common operation and needs to be carefully optimized

- Many algorithms
- No one size fits all

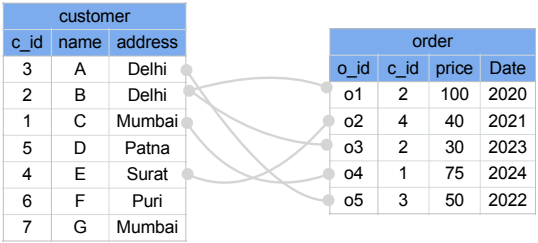
```
SELECT c.name, o.price, o.order_date
FROM customer c, orders o
WHERE c.customer_id = o.customer_id
AND o.price > 50
ORDER BY o.order_date DESC;
```



Logical Plan

Join Algorithms

1. Nested Loop Join (NLJ)
   - Naive NLJ
   - Block NLJ
   - Index NLJ
2. Sort-Merge Join
3. Hash Join
   - Simple hash join
   - Grace hash join
   - Hybrid hash join



customer

c_id	name	address
3	A	Delhi
2	B	Delhi
1	C	Mumbai
5	D	Patna
4	E	Surat
6	F	Puri
7	G	Mumbai

order

o_id	c_id	price	Date
o1	2	100	2020
o2	4	40	2021
o3	2	30	2023
o4	1	75	2024
o5	3	50	2022

Notations & Cost Factors

- For relation R
  - r a tuple in R
  - b_R blocks
  - n_R tuples
- For Relation S
  - s a tuple in S
  - b_S blocks
  - n_S tuples
- Cost model based on # of I/Os
  - #blocks trasferred (recall: each block requires t_T time)
  - #blocks seeked (recall: each block requires t_S time)

customer

c_id	name	address
3	A	Delhi
2	B	Delhi
1	C	Mumbai
5	D	Patna
4	E	Surat
6	F	Puri
7	G	Mumbai

order

o_id	c_id	price	Date
o1	2	100	2020
o2	4	40	2021
o3	2	30	2023
o4	1	75	2024
o5	3	50	2022

Naive Nested Loop Join

Algorithm

1: **for** $r \in R$ **do**
2: **for** $s \in S$ **do**
3: **if** if r and s match **then**
4: $\text{EMIT}(r \bowtie s)$

customer		
c_id	name	address
3	A	Delhi
2	B	Delhi
1	C	Mumbai
5	D	Patna
4	E	Surat
6	F	Puri
7	G	Mumbai

order			
o_id	c_id	price	Date
o1	2	100	2020
o2	4	40	2021
o3	2	30	2023
o4	1	75	2024
o5	3	50	2022

Cost

- #blocks transferred $= b_R + (n_R \times b_S)$
- #seeks $= b_R + n_R$
- cost $= [b_R + (n_R \times b_S)].t_T + (b_R + b_R).t_S$

Naive Nested Loop Join

Example

- $R : b_R = 100; n_R = 5000$
- $S : b_S = 400; n_S = 10,000$

Cost

- $b_R + (n_R \times b_S) = 100 + (5000 \times 400) = 2,000,100$
- $b_R + n_R = 100 + 5000 = 5010$
- At 4 ms/block seek time and 0.1 ms/block transfer time, total time $\approx$ 3.5min

If we consider S as the outer relation

- $b_S + (n_S \times b_R) = 400 + (10,000 \times 100) = 1,000,400$
- $b_S + n_S = 100 + 5000 = 10,400$
- At 4 ms/block seek time and 0.1 ms/block transfer time, total time $\approx$ 2.3min

Block Nested Loop Join

Algorithm

1: **for** each block $B_R \in R$ **do**
2: **for** each block $B_S \in S$ **do**
3: **for** each $r \in B_R$ **do**
4: **for** each $s \in B_S$ **do**
5: **if** r and s match **then**
6: $\text{EMIT}(r \bowtie s)$

customer		
c_id	name	address
3	A	Delhi
2	B	Delhi
1	C	Mumbai
5	D	Patna
4	E	Surat
6	F	Puri
7	G	Mumbai

order			
o_id	c_id	price	Date
o1	2	100	2020
o2	4	40	2021
o3	2	30	2023
o4	1	75	2024
o5	3	50	2022

Cost

- block transfers: $b_R + (b_R \times b_S)$
- seeks: $2b_R$

Block Nested Loop Join

- Reduces to access to disk
- Smaller relation should be the outer table (why?)
  – Smaller based on number of pages!

Optimizations

- If join attribute of inner relation is a key, terminate outer loop as soon as a match is found
- Cache-conscious inner loop: scan alternatively forward and backward
  (how does this help?)
- Use biggest size of a relation as blocking unit (next slide)

Block Nested Loop Join

Assume that we have B buffer pages

- Use $B-2$ buffer pages for R
- Use one buffer page for S
- Use one buffer page for output

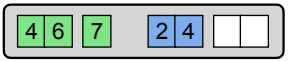
Algorithm

1: **for** each $B-2$ blocks $B'_R \in R$ **do**
2: **for** each block $B_S \in S$ **do**
3: **for** each $r \in B-2$ blocks **do**
4: **for** each $s \in B_S$ **do**
5: **if** r and s match **then**
6: $\text{EMIT}(r \bowtie s)$

- Improved cost $= b_R + (\left\lceil \frac{b_R}{B-2} \right\rceil \times b_S)$
- Improved seeks $= 2 \left\lceil \frac{b_R}{B-2} \right\rceil$

customer		
c_id	name	address
3	A	Delhi
2	B	Delhi
1	C	Mumbai
5	D	Patna
4	E	Surat
6	F	Puri
7	G	Mumbai

order			
o_id	c_id	price	Date
o1	2	100	2020
o2	4	40	2021
o3	2	30	2023
o4	1	75	2024
o5	3	50	2022



Buffer pool (B=4)

and so on...

Block Nested Loop Join

Example

- $R : b_R = 100; n_R = 5000$
- $S : b_S = 400; n_S = 10,000$

Cost

If $B = 12$

- $b_R + (\left\lceil \frac{b_R}{B-2} \right\rceil \times b_S) = 100 + 10 \times 400 = 4,100$
- $2 \left\lceil \frac{b_R}{B-2} \right\rceil = 2 \times 10 = 20$
- At 0.1ms/block transfer time and 4ms/block seek time, total time $\approx$ 0.4s

If $b_R < B - 2$ (outer relation will fit in memory)

- 500 block transfers
- 2 seeks
- At 0.1ms/block transfer time and 4ms/block seek time, total time $\approx$ 0.05s

Index Nested Loop Join

Use an index scan instead of sequential scan for inner relation

Algorithm

1: **for** each $r \in R$ **do**
2: **for** each s $\in$ INDEX$(r_i = s_j)$ **do**
3: **if** r and s match **then**
4: EMIT$(r \bowtie s)$

customer		
c_id	name	address
3	A	Delhi
2	B	Delhi
1	C	Mumbai
5	D	Patna
4	E	Surat
6	F	Puri
7	G	Mumbai

order			
o_id	c_id	price	Date
o1	2	100	2020
o2	4	40	2021
o3	2	30	2023
o4	1	75	2024
o5	3	50	2022

Cost

- b_R block transfers + b_R seeks
- index scan cost: $n_R \times C$
- cost $= b_R(t_T + t_S) + n_R \times C$
  - Assuming C is the cost of each index probe (recall cost of single selection)
- If index on both relations, use one with fewer tuples as outer relation

Sort Merge Join

1. Sorting
   - Sort relations on join key(s)
2. Merge
   - Scan two sorted relations and emit matching tuples
     – Note: This is different than merging of external sort

Sort Merge Join

Algorithm

1: $R' \leftarrow \text{SORT}(R)$, $S' \leftarrow \text{SORT}(S)$ on join keys
2: $p_R \leftarrow$ address of first tuple in R'
3: $p_S \leftarrow$ address of first tuple in S'
4: **while** p_R and p_S **do**
5: **if** $p_R > p_S$ **then**
6: $p_S \leftarrow$ next tuple of S'
7: **if** $p_R < p_S$ **then**
8: $p_R \leftarrow$ next tuple of R'
9: backup if required
10: **if** p_R and p_S match **then**
11: $\text{EMIT}(r \bowtie s)$
12: $p_S \leftarrow$ next tuple of S'

customer		
c_id	name	address
1	C	Mumbai
2	B	Delhi
3	A	Delhi
4	E	Surat
5	D	Patna
6	F	Puri
7	G	Mumbai

order			
o_id	c_id	price	Date
o4	1	75	2024
o1	2	100	2020
o3	2	30	2023
o5	3	50	2022
o2	4	40	2021

Sort Merge Join

Cost

- Sorting R: $2b_R(1 + \left\lceil \log_{B-1} \left\lceil \frac{b_R}{B} \right\rceil \right\rceil)$
- Sorting S: $2b_S(1 + \left\lceil \log_{B-1} \left\lceil \frac{b_S}{B} \right\rceil \right\rceil)$
- Merging: $b_R + b_S$ (block transfers)
- Seeking: $\left\lceil \frac{b_R}{b_B} \right\rceil + \left\lceil \frac{b_S}{b_B} \right\rceil$
  - b_B buffer blocks allocated to each relation

Sort Merge Join

Example

- R : $b_R = 100$; $n_R = 5000$
- S : $b_S = 400$; $n_S = 10,000$

Cost

- if B=11 and $b_B = 1$
- sorting cost for R : $2 \times 100(1 + \lceil \log_{10} \lceil 100/11 \rceil \rceil) = 400$
- sorting cost for S : $2 \times 400(1 + \lceil \log_{10} \lceil 400/11 \rceil \rceil) = 2,400$
- merging cost $= 500$
- seeking: cost $= 500$
- At 0.1ms/transfer seek time and 4ms/block seek time, total time $\approx$ 2.3s

Sort Merge Join

Notes

- Worst case: when join attribute of all tuples contains the same value
- Useful when table(s) is already sorted in join key
- Useful when sorted output is required

Q: Assume there is a secondary index on join attributes in both relations R and S. Would they be useful?

A: No (Why?)

- But, we can use **hybrid merge join technique**, if one file is sorted and other has a secondary B+tree index on join attributes
  1. merge sorted relations with leaf entries of secondary B+-tree
  2. Sort resulting file on address of the unsorted relations – results in sequential reads of both files now!
- This technique can be extended when both files are unsorted

Basic Idea

Key insight

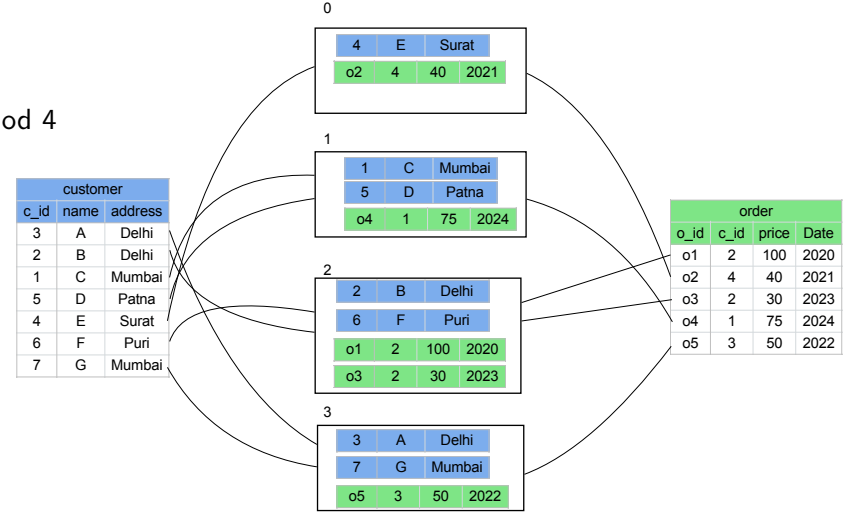
- if $r \in R$ matches $s \in S$, then both tuples when hashed on the join attribute using the same hash function should be in the same bucket i

- Partition R into $R_0, R_1, \ldots R_n$
- Partition S into $S_0, S_1, \ldots S_n$
- Using a hash function h : `join_attribute` $\mapsto \{0, 1, \ldots, n\}$

Basic Idea

Example

- $h : x \bmod 4$



Naive In-memory Hash Join

1. **Build:** Scan the outer relation and build a hash table
2. **Probe:** Scan the inner relation and find the matching tuple

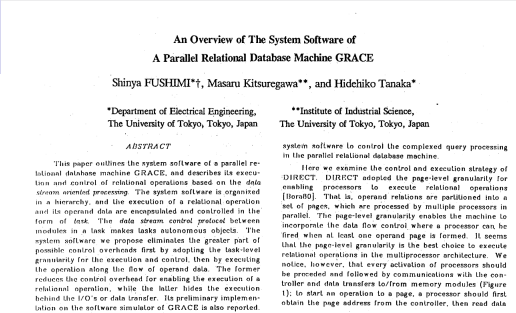
Algorithm

1: build a hash table H for R
2: **for** each $s \in S$ **do**
3: **if** $h(s) \in H$ **then**
4: find matching tuple(s) and EMIT

- Does not work on large relations
- Buffer pool manager may swap out pages of hash table!

Grace Hash Join

- Also known as partitioned hash join
- Adopts a divide and conquer approach
- **Partitioning phase:** Partition R and S using the same hash function
- **Build & Probe Phase:** Compare tuples in each partition



An Overview of The System Software of A Parallel Relational Database Machine GRACE

Shinya FUSHIMI*†, Masaru Kitsuregawa**, and Hidehiko Tanaka*

*Department of Electrical Engineering, The University of Tokyo, Tokyo, Japan

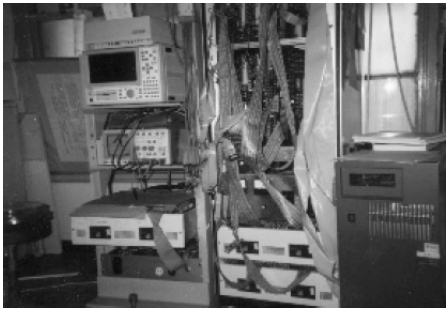
**Institute of Industrial Science, The University of Tokyo, Tokyo, Japan

ABSTRACT

This paper outlines the system software of a parallel relational database machine GRACE, and describes its execution and control of relational operations based on the *data stream oriented processing*. The system software is organized in a hierarchy, and the execution of a relational operation and its operand data are encapsulated and controlled in the form of *task*. The *data stream control* protocol between modules in a task makes tasks autonomous objects. The system software we propose eliminates the greater part of possible control overheads first by adopting the *task-level granularity* for the execution and control, then by executing the operation along the flow of operand data. The former reduces the control overhead for enabling the execution of a relational operation, while the latter hides the execution behind the I/O's or data transfer. Its preliminary implementation on the software simulator of GRACE is also reported. In addition, the novel virtual space management algorithm is system software to control the complexed query processing in the parallel relational database machine.

Here we examine the control and execution strategy of DIRECT. DIRECT adopted the page-level granularity for enabling processors to execute relational operations [Ito80]. That is, operand relations are partitioned into a set of pages, which are processed by multiple processors in parallel. The page-level granularity enables the machine to incorporate the data flow control where a processor can be fired when at least one operand page is formed. It seems that the page-level granularity is the best choice to execute relational operations in the multiprocessor architecture. We notice, however, that every activation of processors should be preceded and followed by communications with the controller and data transfers to/from memory modules (Figure 1): to start an operation to a page, a processor should first obtain the page address from the controller, then read data

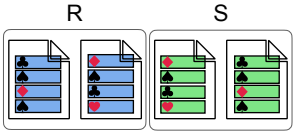
GRACE Parallel Relational Database Machine

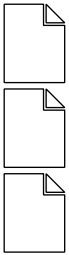


https://museum.ipsj.or.jp/en/computer/other/0014.html

Grace Hash Join (Example)

- Compute $R \bowtie S$



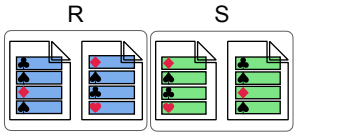


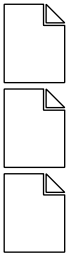
1 Buffer page to read

B-1 Buffer pages for partitioning

Grace Hash Join (Example)– Partitioning Phase

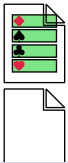
- Partition S





1 Buffer page to read

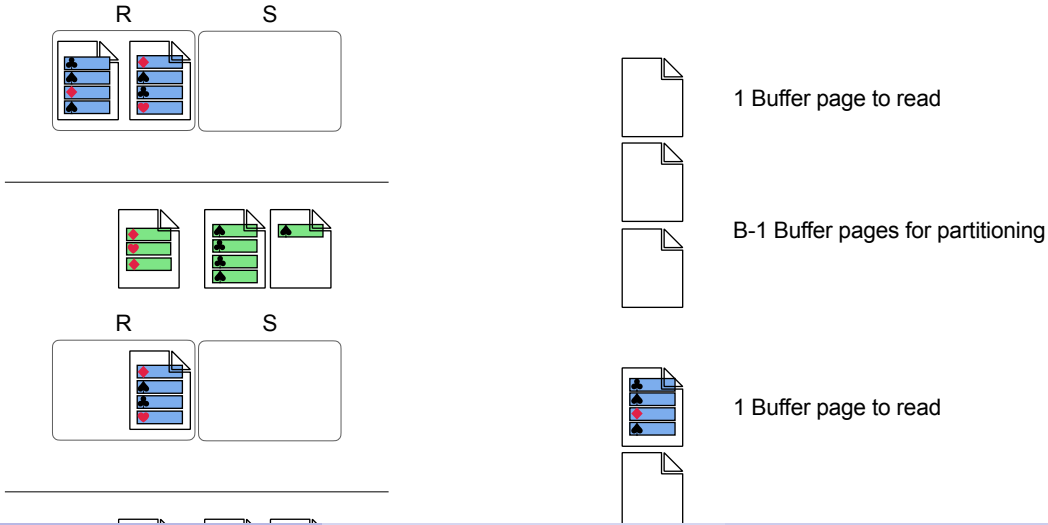
B-1 Buffer pages for partitioning



1 Buffer page to read

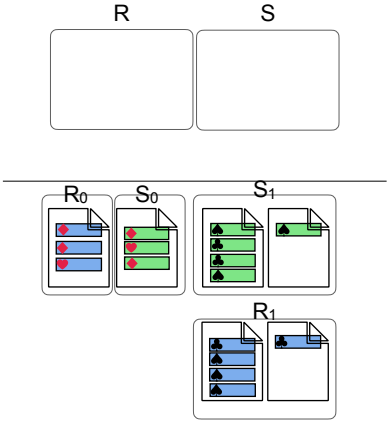
Grace Hash Join (Example)– Partitioning Phase

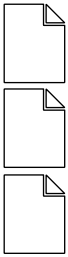
- Partition R



Grace Hash Join (Example)

- After partitioning R and S, we have



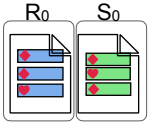


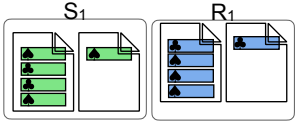
1 Buffer page to read

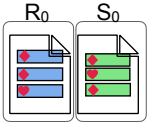
B-1 Buffer pages for partitioning

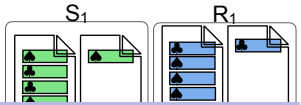
Grace Hash Join (Example) – Build & Probe Phase

- Build in-memory hash table on R

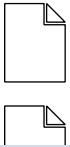




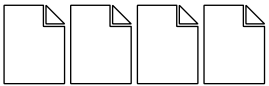




1 Buffer page to read

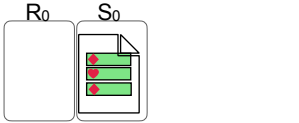


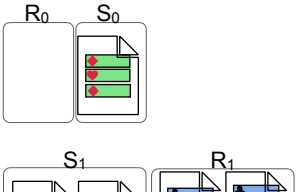
B-2 Buffer pages hash table

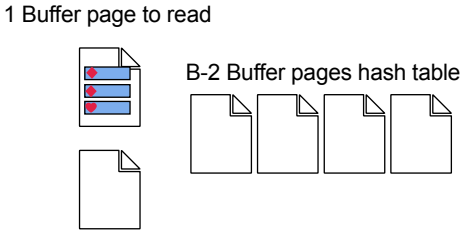


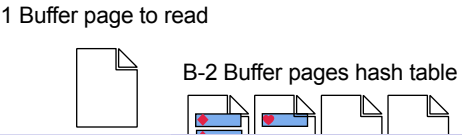
Grace Hash Join (Example) – Build & Probe Phase

- Build in-memory hash table on R



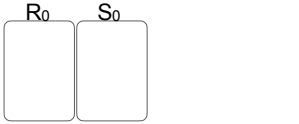


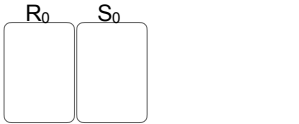


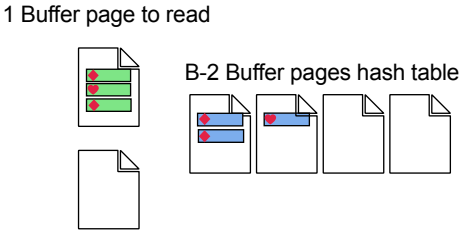


Grace Hash Join (Example) – Build & Probe Phase

- Probe S and compute matching tuples

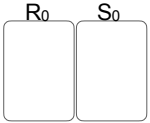


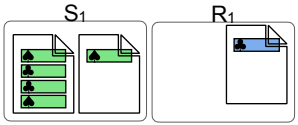


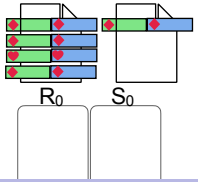


Grace Hash Join (Example) – Build & Probe Phase

- Same process for other partitions

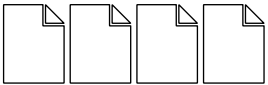






1 Buffer page to read

B-2 Buffer pages hash table



1 Buffer page for output

1 Buffer page to read

Corner Cases

If $n \geq \#$ blocks of memory

- Relations cannot be partitioned in one pass
- in one pass, a relation can at most be partitioned into $B - 1$ partitions (or number of buffers available as output buffers)
- if a partition does not fit in memory, then **recursively partition** using different hash functions

If partitioning is **skewed**

- Increase the number of partitions by a **fudge factor** (usually 20%)
- **Overflow resolution** by re-partitioning
- **Overflow avoidance** by first create many smaller partitions, then combine them to fit in memory
- if resolution and avoidance fail, then use NLJ for keys with "many" values

Partitioned Hash Join

Cost (if no recursive partitioning is required)

- partitioning phase : $2 \times (b_R + b_S)$ block transfers
- build & probe phase : $b_R + b_S + 4n$
  - $4n$ is the overhead for partially filled blocks (usually ignored in cost calculations)
- Total transfer cost $= 3(b_R + b_S)$

- $2(\lceil \frac{b_R}{b_B} \rceil + \lceil \frac{b_S}{b_B} \rceil)$ seeks in partitioning phase
- $2n$ seeks in build and probe phase
- Total seek cost $= 2(\lceil \frac{b_R}{b_B} \rceil + \lceil \frac{b_S}{b_B} \rceil) + 2n$

Partitioned Hash Join

Example

- R : $b_R = 100$; $n_R = 5000$
- S : $b_S = 400$; $n_S = 10,000$

Cost

- assuming $b_B = 3$
- $3(100 + 400) = 1500$ block transfers
- $2(\lceil \frac{100}{3} \rceil + \lceil \frac{400}{3} \rceil) = 336$ seeks
- At 0.1ms/block transfer time and 4ms/block seek time, total time $\approx$ 0.6s

Note

- If memory is large enough, simple hash join requires only $b_R + b_S$ block transfers and 2 seeks!

Hash Join

Notes

- We don't care about size of inner table, only outer table needs to fit in memory
- Use static hashing, if size of outer table is known

  otherwise, use dynamic hashing (Recall extendible hashing)

Homework exercise

Join Algorithm	I/O cost (block transferred)	Example
NLJ		
BNLJ		
SMJ		
GHJ		

Other Operators

Self study: aggregation and set operators (B1: Ch 15 (15.6))